

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:10

Total Marks:50

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

*I Mrudhula K (192524335) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. Smart City Surveillance System: CPU, Memory, and I/O Interaction

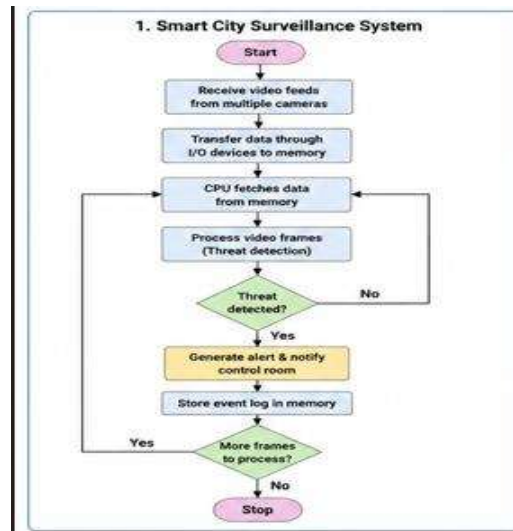
Identification & Analysis of the Bottleneck

The system is suffering from a classic Von Neumann bottleneck exacerbated by high-bandwidth I/O devices (surveillance cameras). In a continuous video feed system, massive arrays of pixels must be moved from the I/O interface into main memory, and then fetched by the CPU for threat-detection algorithms. If the system forces the CPU to manage every byte of this transfer, the CPU spends the majority of its clock cycles waiting on slow I/O operations rather than executing threat-detection logic. This results in severe instruction starvation and critical lag during peak hours.

Comparison Table: Bus Structures

Feature	Single-Bus Architecture	Multi-Bus Architecture (Recommended)
Data Pathway	One shared bus for CPU, Memory, and all I/O devices.	Distinct local buses (CPU-Cache), system buses (CPU-Memory), and expansion buses (I/O).
Bandwidth Contention	Extremely High. Only one transfer can occur at a time.	Low. Allows concurrent transfers across different pathways.
Impact on CPU	CPU must halt instruction execution while I/O writes to memory.	CPU continues executing algorithms from Cache/RAM while I/O transfers data.

Block Diagram Layout (Multi-Bus with DMA)



Recommendation & Justification for System Enhancement

To eliminate the lag, the system must be upgraded to a Multi-bus structure featuring a Direct Memory Access (DMA) controller.

- Justification:** The DMA controller takes over the task of moving video data from the cameras directly into Main Memory. By bypassing the CPU for these massive data dumps, the CPU's instruction execution cycle is completely freed up. The CPU can continuously fetch, decode, and execute the threat-detection algorithms via the system bus without being interrupted by the I/O data stream.

2. Real-Time Stock Trading Application: execution Time and CPI

Evaluation of the Fetch-Decode-Execute Cycle

In high-frequency trading, execution time is dictated by the formula:

$$ExecutionTime = InstructionCount \times CPI \times ClockCycleTime$$

Since the instruction count is necessarily high, the lag is being caused by a high **Cycles Per Instruction (CPI)**. This inflation of CPI occurs during two specific phases of the execution cycle:

- Fetch Phase (Memory Stalls):** The CPU processes data much faster than main memory can provide it. When the CPU has to fetch stock prices directly from RAM, it

wastes multiple clock cycles doing nothing (stalling) while waiting for the data to arrive.

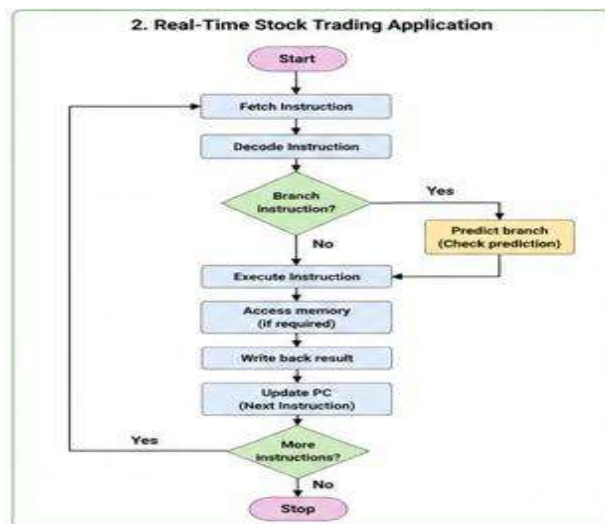
2. **Execute Phase (Branch Penalties):** Trading applications rely heavily on conditional logic (e.g., "If stock drops below X, execute sale"). Modern CPUs try to fetch instructions in advance (pipelining). If a conditional branch goes a different way than the CPU guessed, the entire pipeline must be flushed and restarted, wasting massive amounts of time and drastically increasing the average CPI.

Solution with Justification: Architectural Enhancements

To reduce the CPI and restore real-time processing, I recommend the following CPU design enhancements:

- **Implement an L1/L2 Cache Hierarchy:** By placing small, SRAM-based cache memory directly on the CPU die, frequently accessed stock variables can be fetched in 1-2 clock cycles rather than the 50+ cycles required for main memory. This virtually eliminates fetch-phase stalling.
- **Dynamic Branch Prediction (Branch Target Buffer):** Implement hardware that tracks the historical outcomes of conditional branches. By accurately predicting whether a trade condition will be met before the execution phase, the CPU can keep its instruction pipeline full, minimizing pipeline flushes and reducing the overall CPI.

Block Diagram :



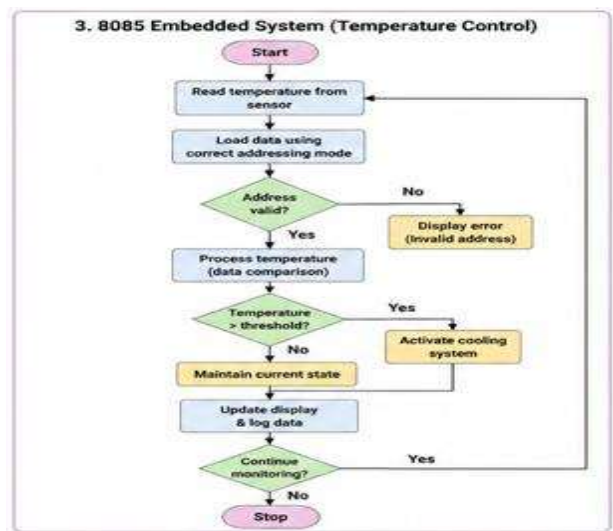
3. 8085 Microprocessor Temperature Control Unit

Analysis of Addressing Modes and the Error

The 8085 microprocessor uses different addressing modes to determine where it should pull data from. The incorrect, occasional static temperature readings indicate that the programmer is likely using **Immediate Addressing** or a flawed static memory reference instead of dynamically reading the active sensor port.

- **Immediate Addressing (MVI A, 32H):** The data is hardcoded into the instruction itself. This will never update.
- **Register Indirect Addressing (MOV A, M):** The instruction tells the CPU to look at a memory address held in a register pair (like HL) and fetch whatever data is currently there.

Flowchart: Reliable Sensor Data Polling



Justification for Instruction Set Usage

To ensure reliable operation, the system must utilize **Direct I/O Addressing** (e.g., IN 01H) or **Register Indirect Addressing**. By forcing the 8085 to actively fetch data from the specific port or memory address mapped to the thermometer during every single loop iteration, the Accumulator is guaranteed to always hold the real-time, physical environment data before making a logic decision.

4.8086 Microprocessor Memory Segmentation Issues

Evaluation of 8086 Segmentation

The 8086 relies on a 20-bit physical address space, achieved by combining a 16-bit segment register with a 16-bit offset using the formula:

$$\text{PhysicalAddress} = (\text{SegmentRegister} \times 16) + \text{Offset}$$

To multitask effectively, memory is divided:

- **Code Segment (CS):** Holds the executable program instructions.
- **Data Segment (DS):** Stores the program variables and static data.
- **Stack Segment (SS):** A LIFO structure used for temporary storage, passing parameters, and storing return addresses during subroutines.
-

Analysis of the Failures

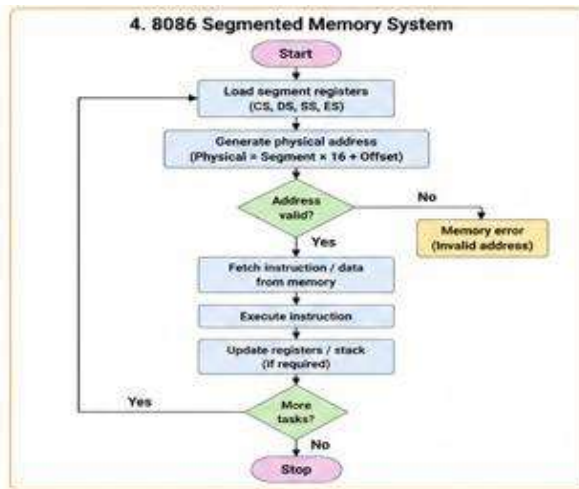
1. **Incorrect Data Access:** In 8086 assembly, the Data Segment is not automatically initialized. If the programmer fails to load the starting address of the program's data into the DS register at the beginning of the code, the CPU will use whatever garbage value is left in DS to calculate physical addresses, resulting in the retrieval of entirely wrong variables.
2. **Stack Overflow:** The Stack Pointer (SP) decrements as data is pushed onto the stack. If the stack is not allocated enough memory, or if an infinite recursive loop occurs, the SP will decrement past the boundaries of the Stack Segment and begin overwriting the Data Segment or Code Segment.

Management for Correct Execution

To resolve this, instruction execution must be strictly managed:

- **Explicit Initialization:** The very first instructions of the program must load the Data Segment (e.g., MOV AX, @DATA followed by MOV DS, AX).
- **Addressing Mode Discipline:** Programmers must respect default segment associations. For instance, using the Base Pointer (BP) register in an addressing mode inherently defaults to the Stack Segment (SS). Mixing these up without segment override prefixes will corrupt memory.

Flowchart diagram:



5.Data Communication System: Hex/Binary Conversions

The Importance of Number System Conversions

Software engineers program and debug using hexadecimal because it is highly condensed (one hex digit perfectly represents four binary bits, or a nibble). However, the CPU's Arithmetic Logic Unit (ALU) and instruction decoder are composed of logic gates that only process binary (electrical highs and lows). Precise conversion is mandatory; a hex payload is merely a human-readable abstraction of the actual binary machine code.

Impact of Conversion Errors on Execution

If the translation software contains a bug and misinterprets a hexadecimal data payload, the resulting binary string fed into the CPU will be corrupted. If this malformed binary data is accidentally routed to the Instruction Register instead of a data register, the CPU will attempt to decode it as an Opcode. This can cause the CPU to execute a completely random, unintended instruction (such as a system halt, a memory wipe, or a branch to an invalid address), leading to a catastrophic system crash.

Design and Benchmarking Recommendations

- Efficient CPU Design:** The CPU architecture should incorporate hardware-level error detection mechanisms, such as checking Parity Bits or verifying Cyclic Redundancy Checks (CRC) embedded in the data packets. If the hardware detects a discrepancy in

the binary payload before execution, it can trigger an interrupt to drop the packet rather than executing it.

- **Benchmarking:** In real-time systems, software-based hex-to-binary conversion routines must be relentlessly stress-tested using standardized benchmark suites. Benchmarking identifies worst-case execution times and exposes translation edge-case errors under heavy data loads, allowing engineers to optimize the conversion algorithms before deployment.

Flowchart:

